



CortexAI — Backend Revision Guide (Every Concept That Matters)

This is your complete revision sheet for the backend of CortexAI. Every concept, pattern, library trick, and minor detail — explained with **why it exists in YOUR project**.

Table of Contents

1. [Microservices Architecture Concepts](#)
2. [Express.js Deep Concepts](#)
3. [Middleware Chain — The Backbone](#)
4. [express-http-proxy Library](#)
5. [Cookie Mechanism \(cookie-parser\)](#)
6. [CORS — Cross Origin Resource Sharing](#)
7. [Redis — Session Store](#)
8. [MongoDB & Mongoose](#)
9. [Firebase Admin SDK](#)
10. [Authentication Flow — Session-Based Auth](#)
11. [Node.js ES Modules \(`type: "module"` \)](#)
12. [Environment Variables & dotenv](#)
13. [Docker Compose \(Redis Container\)](#)
14. [LangChain + LangGraph](#)
15. [Groq & Google Gemini LLM APIs](#)
16. [Inter-Service Communication \(axios\)](#)
17. [HTTP Headers — Custom Headers](#)
18. [crypto.randomUUID\(\)](#)
19. [Morgan — HTTP Logger](#)
20. [JSON.stringify & JSON.parse](#)
21. [Error Handling Patterns](#)
22. [Security Concepts](#)
23. [Minor But Important Details](#)
24. [Quick Recall Flashcards](#)

1. Microservices Architecture Concepts

What is it?

Instead of one giant server (monolith), you split your app into **small independent services** that each handle one responsibility.

Your project's services:

Service	Port	Responsibility	Has its own DB?
Gateway	8000	Routing, auth validation, proxying	No (uses shared Redis)
Auth	8001	Login, logout, user creation	Yes (auth database)
Chat	8002	Conversations & messages CRUD	Yes (chat database)
Agent	8003	AI processing via LangGraph	Yes (agent database)

Why microservices in YOUR project?

- **Independent scaling:** If AI requests spike, you can scale only the Agent service.
- **Independent deployment:** Fix a bug in Chat without touching Auth.
- **Technology flexibility:** Agent uses LangChain/LangGraph, Auth uses Firebase — they don't interfere.
- **Fault isolation:** If Agent crashes, Chat and Auth keep working.

Key pattern used: API Gateway

- The browser talks to ONLY one URL (localhost:8000).
- The Gateway decides where to forward each request.
- Downstream services are hidden — the browser never directly contacts port 8001, 8002, or 8003.

Why each service has its own MongoDB database:

```
mongodb+srv://...mongodb.net/auth    ← Auth service
mongodb+srv://...mongodb.net/chat    ← Chat service
mongodb+srv://...mongodb.net/agent   ← Agent service
```

Same MongoDB Atlas cluster, but **different database names** at the end of the URL. This is called **database-per-service** pattern — each service owns its data and no other service directly queries it.

2. Express.js Deep Concepts

`express()` — What does it return?

```
const app = express()
```

Returns an **application object** — not a server. It's a function that can handle HTTP requests. The server is created when you call `app.listen()`.

`app.use()` vs `app.get()` vs `app.post()`

Method	What it does	Your usage
<code>app.use(path, handler)</code>	Matches ALL HTTP methods (GET, POST, PUT...) on that path	<code>app.use("/chat", protect, proxy)</code> — matches <code>/chat/anything</code>

Method	What it does	Your usage
	prefix	
<code>app.get(path, handler)</code>	Matches ONLY GET requests on that exact path	<code>app.get("/me", protect, getCurrentuser)</code>
<code>app.post(path, handler)</code>	Matches ONLY POST requests	Used in service routes

Important: `app.use("/chat", ...)` strips the prefix

When Gateway does `app.use("/chat", proxy(CHAT_SERVICE))` :

- Browser requests: `GET /chat/get-conversation`
- Chat Service receives: `GET /get-conversation` (the `/chat` prefix is **stripped**)
- That's why Chat Service routes are defined as `/get-conversation` , not `/chat/get-conversation`

`app.use(express.json())`

```
app.use(express.json())
```

- This is a **body parser** middleware.
- Without it, `req.body` is `undefined` .
- It reads the raw HTTP body, parses it as JSON, and attaches the result to `req.body` .
- Used in Auth, Chat, and Agent services — but **NOT in Gateway** (because Gateway proxies raw requests, not parsing bodies).

router — Express Router

```
const router = express.Router()
router.post("/login", login)
export default router
```

- A mini-app that handles routes.
- `app.use("/", router)` mounts it at the root.
- Keeps route definitions separate from server setup (clean code separation).

3. Middleware Chain — The Backbone

What is middleware?

A function with signature `(req, res, next)`. It can:

1. **Read/modify** the request (`req`)
2. **Send a response** (`res.json(...)`) and stop the chain
3. **Call** `next()` to pass control to the next middleware

Your middleware chain on `/chat` route:

```
app.use("/chat", protect, proxyWithHeader(process.env.CHAT_SERVICE))
```

```
graph LR
    A["Path matcher: Only runs for /chat/* URLs"] --> B["Middleware 2: Validate session, attach req.user"]
    B --> C["Middleware 3: Proxy request to Chat service"]
```

Critical concept: `next()`

```
const protect = async (req, res, next) => {  
  // ... validation ...  
  req.user = JSON.parse(session)  
  next() // ← WITHOUT this, the request HANGS FOREVER  
}
```

- If `protect` doesn't call `next()`, `proxyWithHeader` never runs.
- If `protect` sends a response (`res.status(401).json(...)`) AND calls `next()`, you get "headers already sent" error.
- **Rule:** Either send a response OR call `next()`, never both.

Order matters

```
app.use(cookieParser()) // MUST be before protect  
app.use("/chat", protect, ...) // protect needs req.cookies to exist
```

If `cookieParser()` is registered AFTER the routes, `req.cookies` would be `undefined` inside `protect`.

4. express-http-proxy Library

What it does

Takes an incoming Express request and **forwards it entirely** to another server.

Plain proxy (used for `/auth`):

```
app.use("/auth", proxy(process.env.AUTH_SERVICE))
```

- Request comes in: `POST /auth/login { token: "..."}`
- Proxy strips `/auth`, forwards to:

```
POST http://localhost:8001/login { token: "..."} }
```

- Auth service responds → proxy sends response back to browser
- **No modification** — the request passes through untouched.

Custom proxy with `proxyReqOptDecorator` :

```
proxy(serviceUrl, {  
  proxyReqOptDecorator: (proxyReqOpts, srcReq) => {  
    proxyReqOpts.headers['x-user-id'] = srcReq.user.userid  
    return proxyReqOpts    // MUST return the object  
  }  
})
```

- `proxyReqOptDecorator` fires **before** the request is forwarded.
- `proxyReqOpts` = the outgoing request options (you can modify headers, method, etc.)
- `srcReq` = the original incoming Express `req` object.
- **Must return** `proxyReqOpts` — if you forget, the proxy breaks silently.

Other hooks available (not used in your project but good to know):

- `proxyReqBodyDecorator` — modify the body before forwarding
- `userResDecorator` — modify the response before sending to client
- `filter` — conditionally decide whether to proxy or not

5. Cookie Mechanism (cookie-parser)

What is a cookie?

A small piece of data the server tells the browser to store. The browser then **automatically** sends it with every subsequent request to that domain.

How `cookie-parser` works:

```
app.use(cookieParser())
```

- Reads the `Cookie` HTTP header from incoming requests.
- Parses it into a JavaScript object.
- Attaches it to `req.cookies`.
- Without it: `req.cookies` is `undefined`.

Setting a cookie (Auth Service):

```
res.cookie("session", sessionId, {  
  httpOnly: true,      // JavaScript in browser CANNOT read this cookie  
  secure: false,       // Allow HTTP (not just HTTPS) – dev mode only  
  sameSite: "strict",  // Cookie sent ONLY to same origin  
  maxAge: 604800000    // 7 days in MILLISECONDS  
})
```

Reading a cookie (Gateway):

```
const sessionId = req.cookies?.session // "a1b2c3d4-e5f6-..."
```

Clearing a cookie (Logout):

```
res.clearCookie("session") // Tells browser to delete the cookie
```

Important details:

- `httpOnly: true` → Prevents XSS attacks from stealing the session cookie via `document.cookie`.
- `secure: false` → In production, this MUST be `true` (HTTPS only). Currently `false` for `localhost` development.
- `sameSite: "strict"` → Browser won't send this cookie on cross-site requests (CSRF protection).

- `maxAge` is in **milliseconds** (cookie) but Redis `EX` is in **seconds** — easy to confuse.
-

6. CORS — Cross Origin Resource Sharing

The problem:

- Frontend runs on `http://localhost:5173` (Vite dev server)
- Backend Gateway runs on `http://localhost:8000`
- These are **different origins** (different ports = different origin)
- By default, browsers **block** cross-origin requests.

The solution:

```
app.use(cors({
  origin: process.env.FRONTEND_URL, // "http://localhost:5173"
  credentials: true                 // Allow cookies to be sent
}))
```

What each option does:

Option	Value	Why
<code>origin</code>	<code>"http://localhost:5173"</code>	Only this origin can make requests. Using <code>"*"</code> would block cookies.
<code>credentials</code>	<code>true</code>	Allows the browser to send cookies cross-origin. Without this, the <code>session</code> cookie is never sent.

Why `origin: "*" won't work:`

When `credentials: true`, you **cannot** use wildcard `*` as origin. The browser enforces this rule. You must specify the exact origin.

What CORS actually does (behind the scenes):

The `cors()` middleware adds response headers:

```
Access-Control-Allow-Origin: http://localhost:5173
Access-Control-Allow-Credentials: true
```

These headers tell the browser: "Yes, this origin is allowed to read my response."

Preflight requests:

For POST/PUT/DELETE with JSON bodies, the browser sends an **OPTIONS** request first (called "preflight") to check CORS headers. The `cors()` middleware automatically handles this.

7. Redis — Session Store

What is Redis?

An **in-memory** key-value store. Think of it as a giant JavaScript `Map` that lives in a separate process and survives app restarts (if configured).

Why Redis for sessions (not MongoDB)?

Feature	Redis	MongoDB
Speed	~0.1ms reads (RAM)	~5-20ms reads (disk)
TTL (auto-expire)	Built-in <code>EX</code> flag	Requires TTL index setup
Use case	Temporary, fast-access data	Permanent, structured data

Sessions are **temporary** (expire in 7 days) and checked on **every request** — Redis is perfect.

ioredis library:

```
import ioredis from "ioredis"
const redis = new ioredis(process.env.REDIS_URL) // "redis://localhost:6379"
```

Operations used in your project:

Operation	Code	What it does
SET with TTL	<code>redis.set("session-uuid", jsonString, "EX", 604800)</code>	Store value, auto-delete after 604800 seconds (7 days)
GET	<code>redis.get("session-uuid")</code>	Retrieve value (returns null if expired/missing)
DEL	<code>redis.del("session-uuid")</code>	Manually delete (on logout)

Key format: `session-${sessionid}`

- Prefix `session-` is a **namespace convention**.
- If you later add other Redis data (e.g., rate limits), you'd use `ratelimit-${ip}`.
- This prevents key collisions.

The "EX" flag:

```
redis.set(key, value, "EX", 604800)
//          ▲▲      ▲▲▲▲▲▲
//          |      └─ 604800 seconds = 7 days
//          └─ "EX" = set Expiration in Seconds
//          "PX" = set expiration in Milliseconds
```

Redis stores ONLY strings:

```
// Storing an object – MUST stringify:
redis.set(key, JSON.stringify({ userid, name, email, avatar }))

// Retrieving – MUST parse:
const session = await redis.get(key)
req.user = JSON.parse(session) // Back to JavaScript object
```

8. MongoDB & Mongoose

Mongoose basics used in your project:

Schema — defines document structure:

```
const userSchema = new mongoose.Schema({
  firebaseUid: { type: String, unique: true }, // unique = DB-level constraint
  name: String,                               // shorthand for { type: String }
  email: String,
  avatar: String
}, {
  timestamps: true // Auto-adds createdAt and updatedAt fields
})
```

Model — provides CRUD methods:

```
const User = mongoose.model("User", userSchema)
```

- "User" → Mongoose creates a collection called `users` (lowercase + plural, automatic).
- "Conversation" → collection called `conversations`.
- "Message" → collection called `messages`.

Operations used:

Operation	Code	Why
findOne	<code>User.findOne({ firebaseUid: decoded.uid })</code>	Login check exists
create	<code>User.create({ firebaseUid, name, email, avatar })</code>	Login create user
find + sort	<code>Conversation.find({ userId }).sort({ updatedAt: -1 })</code>	Get conv newer
findByIdAndUpdate	<code>Conversation.findByIdAndUpdate(id, { title }, { new: true })</code>	Update conv title

`{ new: true }` in `findByIdAndUpdate`:

```
Conversation.findByIdAndUpdate(id, { title }, { new: true })  
//  
// Without this: returns the OLD document (before update)  
// With this: returns the NEW document (after update)
```

ObjectId references:

```
conversationId: {  
  type: mongoose.Schema.Types.ObjectId,  
  ref: "Conversation"    // References the Conversation model  
}
```

- `ref` enables `.populate()` (not used in your project yet, but important to know).
- This is how Message documents are linked to Conversation documents.

`timestamps: true :`

Automatically adds:

- `createdAt` — set once when document is created
- `updatedAt` — updated every time the document is modified

MongoDB Atlas connection string:

```
mongodb+srv://username:password@cluster0.p2k1stc.mongodb.net/auth
```

▲▲▲▲

└ Database name

The last segment after `/` is the database name. Each service uses a different one.

9. Firebase Admin SDK

Client SDK vs Admin SDK:

Feature	Client SDK (Frontend)	Admin SDK (Backend)
Runs in	Browser	Node.js server
Auth method	<code>signInWithPopup()</code>	<code>verifyIdToken()</code>
Needs	API key (public)	Service account key (PRIVATE)

Feature	Client SDK (Frontend)	Admin SDK (Backend)
Purpose	User-facing login UI	Server-side token verification

Your backend setup:

```
import admin from "firebase-admin"
import { initializeApp, cert } from "firebase-admin/app"
import serviceAccount from "../serviceAccountKey.json" with { type: "json" }

export const app = initializeApp({
  credential: cert(serviceAccount)
})
```

with { type: "json" } — Import assertion:

This is a Node.js ES Module feature. When importing a `.json` file, you must tell Node it's JSON explicitly. Without this, Node refuses to import it (security measure).

verifyIdToken() :

```
const decoded = await getAuth(app).verifyIdToken(token)
```

- Takes the JWT from the frontend.
- Verifies its **signature** using Firebase's public keys.
- Checks if the token is **expired**.
- Returns decoded payload: `{ uid, name, email, picture, ... }`
- Throws an error if token is invalid/expired.

serviceAccountKey.json :

- Contains a **private key** that proves your server is authorized to use Firebase Admin.
 - Downloaded from Firebase Console → Project Settings → Service Accounts.
 - ⚠️ Must NEVER be committed to git (currently it is — security risk).
-

10. Authentication Flow — Session-Based Auth

Why session-based and not JWT-based?

Feature	Session-based (your project)	JWT-based
Token storage	Server-side (Redis)	Client-side (localStorage/cookie)
Revocation	Easy — just delete from Redis	Hard — JWT valid until expiry
Logout	Instant — delete session	Can't truly invalidate
Scalability	Need shared session store	Stateless — no shared store
Security	Cookie httpOnly = immune to XSS	localStorage = vulnerable to XSS

Your choice: Session-based with Redis as shared store.

The Gateway and Auth service share the same Redis instance, which makes session validation work across services.

The complete auth lifecycle:

LOGIN:

- Firestore token → Auth verifies → User upserted in MongoDB
- Session UUID created → User data stored in Redis with 7-day TTL
- Session UUID set as httpOnly cookie

EVERY REQUEST:

- Browser sends cookie → Gateway reads cookie
- Redis lookup with session-UUID → User data attached to req.user
- Request forwarded with x-user-id header

LOGOUT:

- Cookie read → Redis key deleted → Cookie cleared
- Frontend Redux state set to null → Login modal appears

SESSION EXPIRY:

- After 7 days → Redis auto-deletes the key
- Next request → Redis returns null → Gateway returns 401
- Frontend receives 401 → Should redirect to login (not implemented yet)

11. Node.js ES Modules (type: "module")

In every `package.json` :

```
"type": "module"
```

What this changes:

Feature	CommonJS (default)	ES Modules (your project)
Import	<code>const x = require("x")</code>	<code>import x from "x"</code>
Export	<code>module.exports = x</code>	<code>export default x</code>

Feature	CommonJS (default)	ES Modules (your project)
Named export	<code>exports.foo = foo</code>	<code>export const foo = foo</code>
File extensions	Optional	Required — <code>"./db.js"</code> not <code>"./db"</code>
<code>__dirname</code>	Available	❌ Not available
Top-level <code>await</code>	❌ Not available	✅ Available

Why file extensions matter:

```
import connectDb from "./config/db.js" // ✅ Correct
import connectDb from "./config/db"   // ❌ Error in ES modules
```

ES modules require explicit `.js` extensions. CommonJS resolves them automatically.

12. Environment Variables & dotenv

How `dotenv` works:

```
import dotenv from "dotenv"
dotenv.config() // Reads .env file, loads into process.env
```

- Reads the `.env` file in the **current working directory**.
- Parses each `KEY=VALUE` line.
- Sets `process.env.KEY = "VALUE"`.
- Must be called **before** you use any `process.env.*` values.

Alternative pattern (Agent service):

```
import "dotenv/config" // Auto-loads .env on import – one-liner
```

This is equivalent to `dotenv.config()` but more concise.

Important: All values are STRINGS:

```
process.env.PORT // "8000" ← String, not number
const port = process.env.PORT || 8000 // Works because "8000" is truthy
```

13. Docker Compose (Redis Container)

Your `docker-compose.yml` :

```
services:
  redis:
    image: redis          # Official Redis image from Docker Hub
    ports:
      - 6379:6379        # host_port:container_port
```

What this does:

- Pulls the official `redis` Docker image.
- Starts a Redis server inside a container.
- Maps port `6379` on your Mac to port `6379` inside the container.
- Your Node apps connect to `redis://localhost:6379` — Docker handles the rest.

Commands to know:

```
docker-compose up -d    # Start Redis in background
docker-compose down     # Stop and remove container
docker-compose ps       # Check running containers
```

Why Docker for Redis?

Instead of installing Redis natively on macOS (which requires Homebrew), Docker gives you a clean, isolated Redis instance that's easy to start/stop.

14. LangChain + LangGraph

LangChain — What it is:

A framework for building LLM-powered applications. Your project uses it to:

- Create LLM instances (`ChatGroq` , `ChatGoogleGenerativeAI`)
- Invoke LLMs with structured messages (system + human roles)

LLM invocation pattern:

```
const response = await llm.invoke([
  { role: "system", content: "You are CortexAI..." }, // System prompt
  { role: "human", content: state.prompt }             // User message
])
// response.content = "Redis is an in-memory data store..."
```

LangGraph — What it is:

A state machine framework built on top of LangChain. It lets you define **nodes** (functions) and **edges** (transitions) to create complex AI workflows.

Your LangGraph structure:

State definition (`state.js`):

```
const agentState = Annotation.Root({
  prompt: Annotation(),           // Input: user's question
  aiResponse: Annotation(),       // Output: AI's answer
  agent: Annotation(),            // Internal: routing label
  conversationId: Annotation()    // Passthrough: for message saving
})
```

- `Annotation.Root()` defines the shape of data flowing through the graph.
- Every node receives this state and returns a modified version.

Nodes:

```
workflow.addNode("router", router) // Classifies intent
workflow.addNode("chat", chat)      // General chat agent
workflow.addNode("search", searchagent) // Stub
// ... etc
```

Each node is an async function: `(state) => modifiedState`

Edges:

```
// Fixed edge – always go from __start__ to router:
workflow.addEdge("__start__", "router")

// Conditional edge – router decides which agent runs:
workflow.addConditionalEdges("router", (state) => {
  switch (state.agent) {
    case "chat": return "chat"
    case "search": return "search"
    // ...
  }
}, {
  chat: "chat",
  search: "search",
  // ... (mapping of return values to node names)
})

// Fixed edges – agents go to _end_:
workflow.addEdge("chat", "_end_")
```

Compile & Invoke:

```
const graph = workflow.compile() // Locks the graph, ready to execute
const result = await graph.invoke({ prompt, conversationId })
// result = { prompt, aiResponse, agent, conversationId }
```

Special node: search → chat

```
workflow.addEdge("search", "chat")
```

This means: if the router picks "search", the search agent runs first, then the chat agent runs after it. This is a **chained pipeline** — search finds info, chat formats the response.

15. Groq & Google Gemini LLM APIs

Two LLM providers configured:

```
const groq = new ChatGroq({
  model: "openai/gpt-oss-120b",
  apiKey: process.env.GROQ_API_KEY,
})

const gemini = new ChatGoogleGenerativeAI({
  model: "gemini-2.5-flash"
  // Uses GOOGLE_API_KEY from env automatically
})
```

Model routing:

```
export const getModel = async (agent) => {
  switch (agent) {
    case "chat": return groq
    case "search": return groq
    case "coding": return gemini // Coding uses Gemini
    default: return groq
  }
}
```

Why two models?

- **Groq** — very fast inference, good for chat and search.
 - **Gemini** — better at code generation, used for coding agent.
-

16. Inter-Service Communication (axios)

The Agent → Chat Service call:

```
await axios.post(`${process.env.CHAT_SERVICE}/save-message`, {
  conversationId,
  role: "user",
  content: prompt
})
```

Important: This is a server-to-server call.

- Agent service (port 8003) directly calls Chat service (port 8002).
- This bypasses the Gateway — no cookie, no `protect` middleware, no `x-user-id` header.
- This is fine because it's **internal** — only trusted services can reach each other.
- In production, you'd use a private network to ensure only internal services can make these calls.

Why Agent doesn't save messages itself:

- **Database-per-service** rule: Only Chat service owns the messages database.
 - Agent service should NOT directly connect to Chat's MongoDB.
 - Instead, it calls Chat's HTTP API — this is the microservices way.
-

17. HTTP Headers — Custom Headers

`x-user-id` header:

```
proxyReqOpts.headers['x-user-id'] = srcReq.user.userid
```

The `x-` prefix convention:

- `x-` historically meant "non-standard" or "custom" header.
- Although the convention is deprecated (RFC 6648), it's still widely used.
- Your downstream services know to look for `x-user-id` in `req.headers["x-user-id"]`.

Headers are case-insensitive:

```
req.headers["x-user-id"]    // Works
req.headers["X-User-Id"]    // Also works
req.headers["X-USER-ID"]    // Also works
```

HTTP/1.1 headers are case-insensitive. Node.js lowercases them all internally.

18. `crypto.randomUUID()`

```
import crypto from "crypto"
const sessionid = crypto.randomUUID()
// → "a1b2c3d4-e5f6-7890-abcd-ef1234567890"
```

- Built-in Node.js module — no npm install needed.
- Generates a UUID v4 (128 bits of randomness).
- Probability of collision: ~1 in 5.3×10^{36} — effectively impossible.
- Used as the session key — unpredictable and unguessable.

Why UUID and not MongoDB ObjectId?

- ObjectIds are **sequential** and partially **predictable** (they contain timestamp).
 - UUIDs are **random** — an attacker can't guess another user's session ID.
-

19. Morgan — HTTP Logger

```
import morgan from "morgan"
app.use(morgan("dev"))
```

What it does:

Logs every HTTP request to the console:

```
GET /me 200 12.345 ms - 156
POST /auth/login 200 45.678 ms - 89
GET /chat/get-conversation 401 0.234 ms - 28
```

"dev" format:

```
:method :url :status :response-time ms - :content-length
```

Color-coded by status:

-  Green = 2xx (success)
-  Cyan = 3xx (redirect)
-  Yellow = 4xx (client error)
-  Red = 5xx (server error)

Only on Gateway:

Morgan is only used in the Gateway because that's where all requests enter. Individual services don't need it since all traffic flows through the Gateway first.

20. JSON.stringify & JSON.parse

Used for Redis serialization:

```
// STORING (auth.controller.js):
JSON.stringify({ userid: user._id, name, email, avatar })
// → '{"userid":"6832f5a1...","name":"Dhruv","email":"...","avatar":"..."}'

// RETRIEVING (auth.middleware.js):
JSON.parse(session)
// → { userid: "6832f5a1...", name: "Dhruv", email: "...", avatar: "..." }
```

Why this is needed:

Redis stores **only strings**. You can't store a JavaScript object directly. So:

- Before storing: Object → `JSON.stringify()` → String → Redis
- After reading: Redis → String → `JSON.parse()` → Object

Error handling for parse:

```
try {
  req.user = JSON.parse(session)
} catch (parseErr) {
  return res.status(500).json({ message: "invalid session data" })
}
```

If somehow the Redis value gets corrupted (not valid JSON), `JSON.parse` throws. Your middleware catches this.

21. Error Handling Patterns

Every controller/middleware uses try-catch:

```
export const login = async (req, res) => {
  try {
    // ... business logic ...
    return res.status(200).json({ message: "login successful", user })
  } catch (error) {
    return res.status(500).json({ message: "login error" })
  }
}
```

Why `return res.status()...` :

The `return` prevents code from continuing after sending a response. Without it:

```
res.status(401).json({ message: "unauthorized" })
// Code keeps running! Next line might also try to send a response
// → "Cannot set headers after they are sent" error
```

Status codes used in your project:

Code	Meaning	Where used
200	OK — success	All success responses
401	Unauthorized — no/invalid session	<code>protect</code> middleware
500	Internal Server Error — something crashed	All catch blocks

22. Security Concepts

httpOnly cookies:

```
httpOnly: true
```

JavaScript in the browser **cannot** access this cookie via `document.cookie`. Only the browser's HTTP engine can read/send it. This prevents **XSS attacks** from stealing sessions.

sameSite: "strict":

The cookie is NEVER sent on cross-site requests (e.g., if a malicious site makes a request to your API). This prevents **CSRF attacks**.

Session vs Token in Redis:

Only the **session UUID** is in the cookie. The actual user data (userid, name, email) is in Redis. Even if an attacker reads the network traffic, they get a random UUID, not user data.

⚠ Current security issues in your project:

1. `secure: false` — Cookies sent over HTTP. Change to `true` in production.
 2. `serviceAccountKey.json` committed to git — should be in `.gitignore`.
 3. **API keys in `.env`** committed to git — should be in `.gitignore`.
 4. **No rate limiting** — anyone can spam the API.
 5. **No input validation** — `prompt` is not sanitized before being sent to LLM.
-

23. Minor But Important Details

? . Optional chaining:

```
req.cookies?.session    // Returns undefined instead of throwing if cookies is null
selectedConversation?._id // Safe access even if selectedConversation is null
```

|| Default values:

```
const port = process.env.PORT || 8000
// If PORT is undefined/empty, use 8000
```

Mongoose sort() :

```
.sort({ updatedAt: -1 }) // -1 = descending (newest first)
.sort({ createdAt: -1 }) // -1 = descending
.sort({ createdAt: 1 })  // 1 = ascending (oldest first)
```

enum in Mongoose schema:

```
role: {
  type: String,
  enum: ["user", "assistant"] // Only these two values allowed
}
```

If you try to save `role: "admin"`, Mongoose throws a validation error.

ref in Mongoose:

```
conversationId: {
  type: mongoose.Schema.Types.ObjectId,
  ref: "Conversation"
}
```

This doesn't enforce a foreign key (MongoDB has no foreign keys). It's a **hint** for `.populate()` — which replaces the ObjectId with the full document.

unique: true :

```
firebaseUid: { type: String, unique: true }
```

Creates a **unique index** in MongoDB. If you try to create two users with the same `firebaseUid`, MongoDB throws a duplicate key error.

Why EX is seconds but maxAge is milliseconds:

```
// Redis TTL — in SECONDS:  
redis.set(key, value, "EX", 24 * 60 * 60 * 7) // 604800 seconds  
  
// Cookie maxAge — in MILLISECONDS:  
res.cookie("session", id, { maxAge: 24 * 60 * 60 * 1000 * 7 }) // 604800000 ms
```

This is a common source of bugs. If you mix them up:

- Cookie in seconds → expires almost instantly (604800ms = ~10 minutes vs 7 days)
- Redis in milliseconds → would be set for millions of seconds

24. Quick Recall Flashcards

Question	Answer
Why doesn't the browser talk to port 8001 directly?	API Gateway pattern — single entry point at port 8000
What does <code>protect</code> middleware do?	Reads session cookie → Redis lookup → attaches <code>req.user</code> → calls <code>next()</code>
Why <code>httpOnly: true</code> ?	Prevents JavaScript (XSS) from reading the cookie

Question	Answer
Why Redis and not MongoDB for sessions?	Speed (~0.1ms vs ~10ms) + built-in TTL auto-expiration
What does <code>express.json()</code> do?	Parses JSON request body → attaches to <code>req.body</code>
Why <code>credentials: true</code> in CORS?	Without it, browser won't send cookies cross-origin
What does <code>app.use("/chat", ...)</code> strip?	The <code>/chat</code> prefix — downstream service receives <code>/get-conversation</code> not <code>/chat/get-conversation</code>
Why <code>return</code> before <code>res.status()</code> ?	Prevents "headers already sent" error by stopping code execution
What is <code>proxyReqOptDecorator</code> ?	Hook that fires before proxy sends request — lets you modify headers
Why <code>JSON.stringify</code> for Redis?	Redis only stores strings — must serialize objects
What does <code>Annotation.Root()</code> do?	Defines the state schema for a LangGraph state machine
Why Agent calls Chat's API instead of its DB?	Database-per-service rule — only Chat owns the messages database
What does <code>{ new: true }</code> do in <code>findByIdAndUpdate</code> ?	Returns the updated document instead of the old one
Why UUID for session and not ObjectId?	UUIDs are random/unpredictable — ObjectIds are sequential/guessable
What is <code>"EX"</code> in Redis SET?	Sets expiration time in seconds
Why <code>dotenv.config()</code> must be called early?	<code>process.env.*</code> values are undefined until <code>.env</code> is loaded

Question	Answer
What does <code>type: "module"</code> do?	Enables ES Module syntax (<code>import/export</code>) instead of CommonJS (<code>require</code>)
Why <code>.js</code> extension is required in imports?	ES Modules don't auto-resolve extensions like CommonJS does
What does <code>morgan("dev")</code> log?	Every HTTP request: method, URL, status, response time
What's the difference between <code>app.use()</code> and <code>app.get()</code> ?	<code>use()</code> matches all methods + path prefix; <code>get()</code> matches only GET + exact path